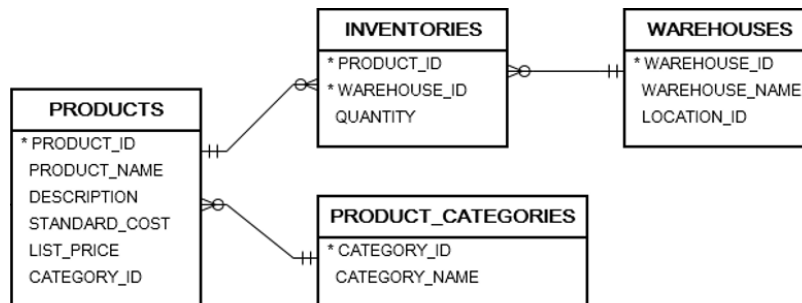


CUESTIONES Y EJERCICIOS

1. Conecta con la base de datos COMPANY y muestra en pantalla los registros de la tabla REGIONS.
2. Conecta con la base de datos COMPANY y muestra en pantalla los registros de la tabla COUNTRIES.
3. Conecta con la base de datos COMPANY y muestra en pantalla el nombre de la región y el nombre del país ordenados por el nombre de región.
4. Crea una clase PruebaInsert que conecte con la base de datos COMPANY y amplíe el inventario de 'Mexico City' para que ahora disponga de 10 unidades de cada uno de los siguientes productos:

PRODUCT_ID	PRODUCT_NAME	DESCRIPTION
7	G.Skill Ripjaws V Series	Speed:DDR4-3200,Type:288-pin DIMM,CAS:15Module:8x8GBSize:64GB
8	Intel Xeon E5-1650 V4	Speed:3.6GHz,Cores:6,TDP:140W
9	Intel Xeon E5-2640 V4	Speed:2.4GHz,Cores:10,TDP:90W

Extracto del modelo de tablas del esquema:



5. Crea una clase PruebaUpdate para que el inventario de 'Mexico City' vea incrementado el stock de todas las CPU que comercializa en 10 unidades.
6. Crea una clase PruebaDelete que conecte con la base de datos COMPANY para que el almacén de 'Mexico City' deje de comercializar cualquier tipo de CPU eliminando estos registros de la tabla de inventario.
7. Modifica el ejercicio 5 para que ahora se use la nueva sintaxis del try-with-resources.

8. Crea una clase **Customer** que permita almacenar los datos de las columnas de la tabla “customers”, para ello debes escribir un constructor con todos los parámetros, el toString, los getters y también preparar la clase para que pueda ser ordenada por el código de cliente y que pueda ser almacenada en un conjunto.

IMPORTANTE: todas las **claves primarias y ajenas** de las clases que representen 1 registro de la BD, **las modelaremos con tipos wrappers** para que puedan almacenar valores nulos. Por ejemplo, en nuestro caso, la columna “customer_id” debe ser modelada con un tipo Integer o Long.

A continuación, escribe una clase **CustomerDataAccess** que tendrá los siguientes métodos:

- *public Customer selectById (int customerId)* debe seleccionar el objeto Customer asociado al registro cuyo código coincida con el que se recibe como parámetro. Si no se encuentra, se devolverá un null.
- *public List<Customer> selectAll ()* debe seleccionar todos los registros de la tabla “customers” y hacer las acciones necesarias para devolver una lista rellena con los objetos Customer resultantes de la conversión de cada registro en un objeto. Si la tabla estuviera vacía se devolverá una lista vacía.
- *public List<Customer> selectByName (String customerName)* debe seleccionar todos los registros de la tabla “customers” cuyo nombre coincida con el que se recibe como parámetro, podría haber más de uno. A continuación, se devolverá una lista rellena con los objetos Customer correspondientes. Si no hubiera resultados se devolverá una lista vacía.

Cada método abrirá, usará y cerrará la conexión con la BD. En el caso de que se produzca una SQLException se debe mostrar su mensaje descriptivo en la consola y lanzar una nueva excepción llamada CustomerDataAccessException con el mensaje descriptivo “Se produjo un error al acceder a los datos del cliente”.

Escribe una clase PruebaCustomerDataAccess que pruebe los tres métodos anteriores. Si se produce una excepción se debe mostrar en la consola su mensaje asociado.

9. A continuación, vamos a mejorar el ejercicio anterior añadiendo los siguientes métodos a la clase CustomerDataAccess:

- *public boolean update (Customer c)* este método debe intentar actualizar el registro de la tabla customers cuya columna “customer_id” sea igual a la correspondiente propiedad del objeto que se recibe como parámetro. Si se consigue se debe devolver *true*, en caso contrario se devolverá *false*.
- *public boolean delete (Customer c)* este método debe intentar borrar el registro de la BBDD asociado al código de cliente del objeto que se recibe como parámetro. Si se consigue se debe devolver *true*, en caso contrario se devolverá *false*.
- *private Customer selectNewestByName (String customerName)* debe seleccionar todos los registros de la tabla “customers” cuyo nombre coincida con el que se recibe

como parámetro ordenando por el `customer_id` de forma descendente. A continuación, se debe devolver el objeto `Customer` correspondiente al más reciente (debería ser el primero del `ResultSet`). Si no hubiera resultados se devolverá un `null`.

- *public Customer create (String name, String address, String website, double creditLimit)* este método debe insertar un nuevo registro en la base de datos con los datos del cliente. A continuación, se utilizará el método auxiliar *selectNewestByName* para obtener el objeto `Customer` asociado al registro que acabamos de insertar.

Cada método abrirá, usará y cerrará la conexión con la BD. En el caso de que se produzca una `SQLException` se debe mostrar su mensaje descriptivo en la consola y lanzar una nueva excepción llamada `CustomerDataAccessException` con el mensaje descriptivo “Se produjo un error al acceder a los datos del cliente”.

Completa la clase `PruebaCustomerDataAccess` para probar los métodos anteriores. Si se produce una excepción se debe mostrar en la consola su mensaje asociado.

10. Añade un nuevo método al ejercicio anterior con la siguiente firma:

```
public List<Customer> selectByCreditLimitRange (double minCreditLimit, double maxCreditLimit, boolean ordered)
```

Este método debe seleccionar los registros correspondientes a los clientes cuyo límite de crédito esté comprendido dentro del rango especificado (se deben incluir los valores extremos). Además, si el parámetro *ordered* es *true* entonces la consulta debe llevar un “ORDER BY `credit_limit`”. En caso contrario, se ejecutará una versión de la consulta sin ordenar.

Por último, se debe devolver la lista de objetos `Customer` correspondiente al `ResultSet` obtenido. Si la consulta no devuelve ningún registro se debe devolver una lista vacía.

Haz una prueba del nuevo método.